NATS Interface Reference

Wire protocol and message schemas for the Cross Guard NATS bus

Overview

Cross Guard relays messages between Mattermost instances using NATS as the transport layer. Every event (post, edit, delete, reaction) is wrapped in a standard JSON envelope and published to a configured NATS subject. The receiving instance subscribes to its inbound subject, deserializes the envelope, and applies the event locally.

Transport	NATS (core publish/subscribe)
Wire Format	JSON (UTF-8 encoded bytes)
Subject Prefix	`crossguard.` (required for all subjects)
Subject Pattern	`crossguard.{domain}`, e.g. `crossguard.high`, `crossguard.low`
Retry Policy	Up to 3 attempts with exponential backoff (500ms base, 5s max)
Concurrency	Up to 50 concurrent outbound publishes per plugin instance

Data Flow

Outbound (Local to Remote)

```
User action in Mattermost
    |
    v
Plugin hook fires (MessageHasBeenPosted, etc.)
    |
    v
Check: channel relay enabled? team linked to outbound connection?
    |
    v
Build Message envelope (type + serialized payload)
```

```
    |
    v
Queue for async publish (semaphore-gated, max 50)
    |
    v
Publish to NATS subject with retry (3x, exponential backoff)
    |
    v
Flush to confirm delivery
```

Inbound (Remote to Local)

```
NATS message arrives on subscribed subject
    |
    v
Deserialize Message envelope
    |
    v
Dispatch by message type (post, update, delete, reaction, test)
    |
    v
Resolve team and channel by name (apply rewrite rules if configured)
    |
    v
Verify team AND channel are linked to this inbound connection
    |
    v
Resolve user (lookup by username or use relay bot)
    |
    v
Create/Update/Delete post or reaction in Mattermost
    |
    v
Store post ID mapping (remote ID to local ID)
    |
    v
Set "crossguard_relayed" property (prevents relay loops)
```

Message Envelope

Every message published to NATS uses this standard envelope. The `type` field determines which payload schema is serialized inside `json`.

NATS **`MessageEnvelope`**

Schema

`type`	**string, required**. One of the message type values. Determines the payload schema.
`timestamp`	**string, required**. UTC timestamp in RFC 3339 format (e.g. `2026-03-27T14:30:00Z`). Set when the envelope is created.
`json`	**string, required**. JSON-encoded payload. Deserialize according to `type` to obtain the inner message object.

Example

```
{
  "type": "crossguard_post",
  "timestamp": "2026-03-27T14:30:00Z",
  "json": "{\"post_id\":\"8swgtrrdiff89yj1z33pofo15r\",\"channel_id\":\"9a1f4h6e
}
```

Message Types

The `type` field in the envelope acts as a discriminator. Each value maps to a specific payload schema serialized in the `json` field.

Type Value	Go Constant	Payload Schema	Descrip
`crossguard_post`	`MessageTypePost`	PostMessage	A new p was crea
`crossguard_update`	`MessageTypeUpdate`	PostMessage	An existi post was

Type Value	Go Constant	Payload Schema	Descrip
			edited
`crossguard_delete`	`MessageTypeDelete`	DeleteMessage	A post w deleted
`crossguard_reaction_add`	`MessageTypeReactionAdd`	ReactionMessage	A reacti was add to a pos
`crossguard_reaction_remove`	`MessageTypeReactionRemove`	ReactionMessage	A reacti was removec from a p
`crossguard_test`	`MessageTypeTest`	TestMessage	Connect test messag

PostMessage

Used for both `crossguard_post` (new posts) and `crossguard_update` (edits). For updates, the fields reflect the post state after the edit.

PAYLOAD **PostMessage**

Fields

Field	Type	Required	Description
`post_id`	string	Yes	Unique identifier of the post on the originating server
`root_id`	string	No	ID of the root post if this is a threaded reply. Omitted for top-level posts.
`channel_id`	string	Yes	ID of the channel on the originating server

Field	Type	Required	Description
`channel_name`	string	Yes	URL-safe channel name (used for cross-domain resolution)
`team_id`	string	Yes	ID of the team on the originating server
`team_name`	string	Yes	URL-safe team name (used for cross-domain resolution, subject to rewrite rules)
`user_id`	string	Yes	ID of the post author on the originating server
`username`	string	Yes	Username of the post author
`message`	string	Yes	Markdown-formatted message body
`create_at`	integer (int64)	Yes	Unix timestamp in milliseconds when the post was originally created

Example: New Post

```
{
  "post_id": "8swgtrrdiff89yj1z33pofo15r",
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
  "team_name": "test-a",
  "user_id": "7m4k9j2p63ynfr8x1qwt5hab4e",
  "username": "alice",
  "message": "Hello from the other side",
  "create_at": 1711547400000
}
```

Example: Threaded Reply

```
{
  "post_id": "kf7g2m4x91yrnh5w3pqs8jab6e",
  "root_id": "8swgtrrdiff89yj1z33pofo15r",
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
  "team_name": "test-a",
  "user_id": "3n6k8j1p42ygfr9x7qwt5hab2c",
```

```
  "username": "bob",
  "message": "Replying in thread",
  "create_at": 1711547460000
}
```

DeleteMessage

Identifies a post that was deleted on the originating server. The receiving instance uses the `post_id` to look up the local copy via its post ID mapping and delete it.

PAYLOAD `DeleteMessage`

Fields

Field	Type	Required	Description
`post_id`	string	Yes	Unique identifier of the deleted post on the originating server
`channel_id`	string	Yes	ID of the channel the post belonged to
`channel_name`	string	Yes	URL-safe channel name
`team_id`	string	Yes	ID of the team on the originating server
`team_name`	string	Yes	URL-safe team name

Example

```
{
  "post_id": "8swgtrrdiff89yj1z33pofo15r",
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
```

```
  "team_name": "test-a"
}
```

ReactionMessage

Represents a reaction added to or removed from a post. Used by both `crossguard_reaction_add` and `crossguard_reaction_remove`. The receiving instance resolves the local post via its post ID mapping and applies or removes the reaction.

PAYLOAD **ReactionMessage**

Fields

Field	Type	Required	Description
`post_id`	string	Yes	ID of the post the reaction applies to
`channel_id`	string	Yes	ID of the channel containing the post
`channel_name`	string	Yes	URL-safe channel name
`team_id`	string	Yes	ID of the team on the originating server
`team_name`	string	Yes	URL-safe team name
`user_id`	string	Yes	ID of the user who added or removed the reaction
`username`	string	Yes	Username of the user
`emoji_name`	string	Yes	Emoji name without colons (e.g. `thumbsup`, not `:thumbsup:`)

Example

```
{
  "post_id": "8swgtrrdiff89yj1z33pofo15r",
```

```
  "channel_id": "9a1f4h6e17yfmk9w6pxs3jqd3e",
  "channel_name": "town-square",
  "team_id": "5f3h8k2m41ygdj7w8pxr6nqb9c",
  "team_name": "test-a",
  "user_id": "7m4k9j2p63ynfr8x1qwt5hab4e",
  "username": "alice",
  "emoji_name": "thumbsup"
}
```

TestMessage

A lightweight message used to verify NATS connectivity. Sent via the `POST /api/v1/test-connection` REST endpoint when testing an outbound connection. The receiving instance logs receipt and confirms the connection is functional.

PAYLOAD **TestMessage**

Fields

Field	Type	Required	Description
`id`	string	Yes	A Mattermost-generated unique identifier for the test request

Example

```
{
  "id": "dj7f8k2m41ygsr9w6pxt3nqb4e"
}
```

Complete Wire Format Examples

Each example below shows the full envelope as it appears on the NATS wire, with the `json` field containing the serialized payload string.

New Post

```
{
  "type": "crossguard_post",
  "timestamp": "2026-03-27T14:30:00Z",
  "json": "{\"post_id\":\"8swgtrrdiff89yj1z33pofo15r\",\"channel_id\":\"9a1f4h6e17yfm
}
```

Post Edit

```
{
  "type": "crossguard_update",
  "timestamp": "2026-03-27T14:31:00Z",
  "json": "{\"post_id\":\"8swgtrrdiff89yj1z33pofo15r\",\"channel_id\":\"9a1f4h6e17yfm
}
```

Post Delete

```
{
  "type": "crossguard_delete",
  "timestamp": "2026-03-27T14:32:00Z",
  "json": "{\"post_id\":\"8swgtrrdiff89yj1z33pofo15r\",\"channel_id\":\"9a1f4h6e17yfm
}
```

Reaction Added

```
{
  "type": "crossguard_reaction_add",
  "timestamp": "2026-03-27T14:33:00Z",
  "json": "{\"post_id\":\"8swgtrrdiff89yj1z33pofo15r\",\"channel_id\":\"9a1f4h6e17yfm
}
```

Reaction Removed

```
{
  "type": "crossguard_reaction_remove",
  "timestamp": "2026-03-27T14:34:00Z",
  "json": "{\"post_id\":\"8swgtrrdiff89yj1z33pofo15r\",\"channel_id\":\"9a1f4h6e17yfm
}
```

Connectivity Test

```
{
  "type": "crossguard_test",
  "timestamp": "2026-03-27T14:35:00Z",
  "json": "{\"id\":\"dj7f8k2m41ygsr9w6pxt3nqb4e\"}"
}
```

Connection Configuration

Each NATS connection (inbound or outbound) is configured in the System Console. This schema is stored in plugin settings and is not transmitted over the wire. It is included here for reference when setting up integrations.

CONFIG `NATSConnection`

Fields

Field	Type	Required	Description
`name`	string	Yes	Connection identifier. Lowercase alphanumeric and hyphens only (pattern: `^[a-z0-9-]+$`).
`address`	string	Yes	NATS server address (e.g. `nats://nats:4222`).
`subject`	string	Yes	NATS subject. Must start with `crossguard.` (e.g. `crossguard.high`).
`tls_enabled`	boolean	No	Enable TLS encryption. Default: `false`.

Field	Type	Required	Description
`auth_type`	string	No	Authentication method: `none`, `token`, or `credentials`. Default: `none`.
`token`	string	Conditional	Bearer token. Required when `auth_type` is `token`.
`username`	string	Conditional	Username. Required when `auth_type` is `credentials`.
`password`	string	Conditional	Password. Required when `auth_type` is `credentials`.
`client_cert`	string	No	Filesystem path to the client TLS certificate.
`client_key`	string	No	Filesystem path to the client TLS private key.
`ca_cert`	string	No	Filesystem path to the CA certificate for server verification.

Example: Token Authentication

```
{
  "name": "high-side",
  "address": "nats://nats:4222",
  "subject": "crossguard.high",
  "tls_enabled": false,
  "auth_type": "token",
  "token": "my-secret-token"
}
```

Example: TLS with Credentials

```
{
  "name": "secure-relay",
  "address": "nats://nats.example.com:4222",
  "subject": "crossguard.relay",
  "tls_enabled": true,
  "auth_type": "credentials",
  "username": "crossguard-svc",
  "password": "s3cret",
```

```
    "client_cert": "/etc/crossguard/client.crt",
    "client_key": "/etc/crossguard/client.key",
    "ca_cert": "/etc/crossguard/ca.crt"
  }
```

Connection Behavior

Parameter	Value	Description
Connect Timeout (test)	30 seconds	One-time connections used by the test-connection API endpoint
Connect Timeout (relay)	5 seconds	Persistent relay connections established at plugin activation
Max Reconnects	Unlimited	Relay connections attempt reconnection indefinitely
Reconnect Wait	2 seconds	Delay between reconnection attempts
Publish Retries	3	Maximum attempts per message publish
Publish Base Delay	500ms	Initial backoff delay between retries
Publish Max Delay	5 seconds	Maximum backoff delay between retries
Concurrent Publishes	50	Maximum concurrent outbound publish operations (semaphore-gated)

Loop Prevention

Cross Guard prevents infinite relay loops through multiple mechanisms:

- **Relayed property:** Every post created by inbound message processing is tagged with a `crossguard_relayed` post property. Outbound hooks skip posts with this property.

- **System message filtering:** System-generated messages (join/leave, header changes, etc.) are never relayed.
- **Bot post filtering:** Posts from the relay bot itself are excluded from outbound processing.
- **Connection linkage validation:** Messages are only published to outbound connections that are explicitly linked to the originating team. Inbound messages are only processed if the target team and channel are linked to the receiving inbound connection.

> **OpenAPI Specification**
> The machine-readable OpenAPI 3.1 specification for this interface is available at `assets/nats-api.yaml` in the plugin source repository.